

# The mathematical and functional flow of modern Transformer-based architectures

A formula cheat-sheet: standard SDPA, sub-quadratic attention and inference

March 2026

## 1 Mathematical Preliminaries and Notation

To ensure no prior knowledge is assumed, the following foundational operators and sets are defined:

- $\mathbb{R}^{a \times b}$ : A matrix of real numbers with  $a$  rows and  $b$  columns[cite: 1]. This represents a 2D grid of continuous values.
- $\odot$ : Element-wise multiplication (Hadamard product)[cite: 3]. If  $C = A \odot B$ , then  $C_{i,j} = A_{i,j} \times B_{i,j}$ [cite: 4]. Unlike standard matrix multiplication, this scales corresponding values directly against each other. x
- $\text{Linear}(x)$ : A standard linear transformation defined as  $\text{Linear}(x) = xW + b$ , where  $W$  is a learnable weight matrix and  $b$  is a learnable bias vector[cite: 7]. This is the basic building block of neural transformations, projecting data from one dimensional space to another.

—

## 2 The Input and Embedding Layer

Neural networks require continuous numerical representations to process discrete text[cite: 8]. You cannot multiply the word "apple" by a weight matrix; it must become an array of numbers.

### 2.1 The Embedding Matrix

The embedding weight matrix  $W_e$  is a learnable dictionary mapping discrete words to continuous vectors[cite: 9]. Think of it as a massive lookup table where every row corresponds to a specific word's "meaning" in space.

$$W_e \in \mathbb{R}^{N \times d} \tag{1}$$

**Definitions:**

- $N$ : Vocabulary Size (the total number of unique words/tokens the model knows)[cite: 10].
- $d$ : Embedding Dimension (the size of the continuous vector representing each token)[cite: 11]. Larger dimensions allow the model to capture more nuanced nuances (e.g., gender, tense, sentiment).

## 2.2 Input Transformation

An input sequence of  $c$  tokens is represented as a matrix of one-hot vectors,  $O$ [cite: 12]. A one-hot vector contains a 1 at the index of the specific word, and 0s everywhere else[cite: 13].

$$E = O \cdot W_e \in \mathbb{R}^{c \times d} \quad (2)$$

### Definitions and Mechanics:

- $O \in \mathbb{R}^{c \times N}$ : The one-hot encoded input matrix[cite: 14].
- $c$ : Context Size (sequence length). The number of tokens currently being processed[cite: 15].
- $E \in \mathbb{R}^{c \times d}$ : The dense embedding matrix[cite: 16]. Mathematically, multiplying the one-hot matrix  $O$  by  $W_e$  simply "plucks out" the correct rows from  $W_e$ , assembling them into a sequence of continuous vectors.

## 2.3 Positional Encoding (For Attention Models)

**Concept:** Because standard Transformers process all  $c$  tokens in parallel, we must inject sequence information so the model knows the order of words[cite: 16]. Without this, "The dog bit the man" and "The man bit the dog" would look mathematically identical. We add a Positional Encoding matrix  $P$  to the embeddings:

$$Z_0 = E + P \in \mathbb{R}^{c \times d} \quad (3)$$

Elements in  $P$  are calculated using sine (sin) and cosine (cos) functions of varying frequencies:

$$P_{(pos, 2i)} = \sin\left(\frac{pos}{10000^{2i/d}}\right) \quad (4)$$

$$P_{(pos, 2i+1)} = \cos\left(\frac{pos}{10000^{2i/d}}\right) \quad (5)$$

### Definitions and Mechanics:

- $pos$ : Token position index (0 to  $c - 1$ )[cite: 17].
- $i$ : Dimension index along the vector (0 to  $d/2$ )[cite: 18].
- **Why Sine/Cosine?** The term  $10000^{2i/d}$  creates a geometric progression of wavelengths. Early dimensions in the vector fluctuate wildly (like the seconds hand on a clock), while later dimensions fluctuate very slowly (like the hour hand). This allows the model to easily calculate relative distances between words.
- $Z_0 \in \mathbb{R}^{c \times d}$ : The initial sequence matrix that will be passed into the very first architectural block[cite: 19].

—

## 3 Standard Multi-Head Attention (MHA)

This block constitutes the core routing mechanism of the Transformer, allowing tokens to share information across the context window  $c$ [cite: 20].

### 3.1 Defining the Input Sequence ( $Z$ )

Let  $Z \in \mathbb{R}^{c \times d}$  represent the continuous vector sequence entering the current block[cite: 21].

- For the very first block,  $Z$  is exactly the embedded and positionally encoded input:  $Z = Z_0$ [cite: 22].
- For any subsequent block  $l$  (where  $l$  goes up to  $k$ ),  $Z$  is the final output of the previous block:  $Z = Z_{l-1}$ [cite: 23].

### 3.2 Linear Projections: Queries, Keys, and Values

We split the embedding dimension  $d$  into  $h$  "heads" to learn different relationships simultaneously[cite: 24]. For example, one head might look for grammatical subjects, while another looks for emotional tone.

$$Q_i = ZW_i^Q, \quad K_i = ZW_i^K, \quad V_i = ZW_i^V \in \mathbb{R}^{c \times d_k} \quad (6)$$

**Definitions and The Retrieval Analogy:**

- $h$ : Number of attention heads[cite: 25].
- $d_k = d/h$ : Dimension of each head.
- $W_i^Q, W_i^K, W_i^V \in \mathbb{R}^{d \times d_k}$ : Trainable weight matrices mapping the input  $Z$  to Queries, Keys, and Values[cite: 26].
- **Query ( $Q$ ):** What a token is looking for (e.g., "I am an adjective looking for my noun").
- **Key ( $K$ ):** What a token is (e.g., "I am a noun describing an animal").
- **Value ( $V$ ):** The token's actual underlying meaning/content that will be transferred if a Query and Key match.

### 3.3 Scaled Dot-Product Attention and Softmax

$$\text{head}_i = \text{Softmax} \left( \frac{Q_i K_i^T}{\sqrt{d_k}} \right) V_i \in \mathbb{R}^{c \times d_k} \quad (7)$$

**Definitions and Mechanics:**

- $Q_i K_i^T \in \mathbb{R}^{c \times c}$ : Raw similarity scores. We take the dot product of every Query against every Key. A high dot product means high relevance. This requires  $c \times c$  calculations, creating an  $O(c^2)$  computational bottleneck[cite: 27].
- $\sqrt{d_k}$ : Scaling factor[cite: 28]. If  $d_k$  is large, dot products can yield massive numbers. This pushes the Softmax function into regions where gradients approach zero (vanishing gradients). Dividing by  $\sqrt{d_k}$  preserves variance and keeps learning stable.
- **Softmax:** Normalizes the  $c \times c$  similarity scores into a probability distribution summing to 1.

$$\text{Softmax}(x_i) = \frac{e^{x_i}}{\sum_j e^{x_j}} \quad (8)$$

- The normalized scores are then multiplied by the Values ( $V_i$ ). This creates a weighted sum: tokens absorb a lot of information from highly relevant tokens, and zero information from irrelevant ones.

### 3.4 Multiple-Head Concatenation

$$\text{MHA}(Z) = [\text{head}_1 \parallel \dots \parallel \text{head}_h] \cdot W^O \in \mathbb{R}^{c \times d} \quad (9)$$

**Definitions:**

- $\parallel$ : The concatenation operator (joining matrices side-by-side)[cite: 29]. We paste all the individual heads back together.
- $W^O \in \mathbb{R}^{d \times d}$ : The trainable output projection matrix that mixes the heads together[cite: 30], synthesizing the different perspectives learned by the parallel heads.

—

## 4 Sub-Quadratic Alternatives (Linear Time $O(c)$ )

To avoid the  $O(c^2)$  bottleneck of  $Q_i K_i^T$ , modern architectures compress the sequence history into a fixed-size mathematical state[cite: 31].

### 4.1 Integration into the Mathematical Flow

In these architectures, the  $O(c^2)$  Multi-Head Attention block is completely removed[cite: 32]. However, the rest of the network’s structure remains identical. The sub-quadratic module simply replaces  $\text{MHA}(Z)$ , accepting the identical input  $Z \in \mathbb{R}^{c \times d}$  and returning an output of the exact same dimensions  $\mathbb{R}^{c \times d}$ [cite: 33]. To process  $Z$  in linear time, these modules read the matrix  $Z$  row-by-row sequentially[cite: 34]. Let  $x_t \in \mathbb{R}^{1 \times d}$  represent the specific row (token vector) of  $Z$  at sequence position  $t$  (where  $t$  goes from 1 to  $c$ )[cite: 35].

### 4.2 State Space Models (SSMs) like Mamba

**Concept:** SSMs track a compressed hidden state  $h_t$  over time[cite: 36]. Instead of looking back at all previous tokens, the model updates this singular state vector with each new token. For the token vector  $x_t$  at step  $t$ :

$$h_t = \bar{A}h_{t-1} + \bar{B}_t x_t, \quad y_t = C_t h_t \quad (10)$$

**The Selective Mechanism:** Traditional SSMs use static matrices. Mamba introduces a *selective mechanism* by making the matrices data-dependent (changing based on the current token  $x_t$ ).

$$\Delta_t = \text{Softplus}(\text{Linear}(x_t)), \quad \bar{B}_t = \text{Linear}(x_t), \quad C_t = \text{Linear}(x_t) \quad (11)$$

**Definitions and Mechanics:**

- $h_t \in \mathbb{R}^n$ : The hidden state vector.  $n$  is the State Expansion Dimension, which determines the capacity of the memory[cite: 37].
- $\bar{A} \in \mathbb{R}^{n \times n}$ : The state transition matrix[cite: 38]. It controls how the previous hidden state  $h_{t-1}$  decays over time.
- $\bar{B}_t \in \mathbb{R}^{d \times n}$ : The input projection matrix. Because it is parameterized by  $x_t$ , the model learns *what* information from the current token is worth memorizing.
- $C_t \in \mathbb{R}^{n \times d}$ : The output projection matrix. It reads the hidden state  $h_t$  and extracts the relevant features to produce the current output  $y_t$ .

- $\Delta_t \in \mathbb{R}^d$ : The discrete step size. This acts as an information gate. A large  $\Delta_t$  forces the model to focus heavily on the new input  $x_t$  and update the state. A small  $\Delta_t$  allows the model to ignore  $x_t$  and preserve the historical context within  $h_{t-1}$ . *Crucially,  $\Delta_t$  determines the continuous-to-discrete transformation*: the continuous baseline matrices  $A$  and  $B$  are integrated into discrete matrices  $\bar{A}$  and  $\bar{B}_t$  using rules like  $\bar{A} = \exp(\Delta_t A)$  and  $\bar{B}_t \approx \Delta_t B_t$ . This scales the "force" of the current token update.
- **Softplus( $x$ )**: A smooth, strictly positive activation function defined mathematically as  $\log(1 + e^x)$ [cite: 39]. It ensures the discretization step  $\Delta_t$  is always positive, preventing numerical instability during integration.

**Routing to the FFN:** Because the SSM operates token-by-token, the discrete outputs  $y_t \in \mathbb{R}^{1 \times d}$  produced at each step  $t$  are collected and stacked vertically. This forms the full sequence output matrix  $Y = [y_1, y_2, \dots, y_c]^T \in \mathbb{R}^{c \times d}$ . This matrix  $Y$  explicitly replaces the  $\text{MHA}(Z)$  term in the network pipeline. It will be passed forward into the residual and Layer Normalization functions (detailed in Section 5.4) to construct  $Z'$ , which ultimately serves as the input to the Feed-Forward Network.

### 4.3 Gated Linear Attention (DeltaNet)

**Concept:** Computes  $Q(K^T V)$  instead of  $(QK^T)V$  to avoid  $c \times c$  scaling, using a running memory  $S_t$ [cite: 40]. By changing the order of matrix multiplication (associative property), the model maintains a fixed-size memory rather than a growing context window. For token  $x_t$ :

$$g_t = \sigma(\text{Linear}(x_t)), \quad S_t = g_t \odot S_{t-1} + k_t v_t^T, \quad o_t = q_t S_t \quad (12)$$

#### Definitions and Mechanics:

- $S_t \in \mathbb{R}^{d_k \times d_v}$ : The fixed-size memory matrix[cite: 41]. This stores the summarized context of all tokens up to step  $t$ .
- $k_t \in \mathbb{R}^{d_k}$  and  $v_t \in \mathbb{R}^{d_v}$ : The current token's Key and Value vectors.
- $k_t v_t^T$ : The outer product of the Key and Value. This produces a  $d_k \times d_v$  matrix representing the new information to be added to the memory state.
- $\sigma(x)$ : The Sigmoid function, defined as  $\sigma(x) = \frac{1}{1+e^{-x}}$ [cite: 42]. It squeezes inputs into a range between  $(0, 1)$  to act as a fractional "forget gate"  $g_t$ , allowing the memory to decay over time[cite: 43]. If  $g_t$  is close to 0, old memory is wiped. If  $g_t$  is close to 1, old memory is fully retained.
- $q_t \in \mathbb{R}^{d_k}$ : The current token's Query vector.
- $o_t = q_t S_t \in \mathbb{R}^{1 \times d_v}$ : The output for the current step. The Query vector essentially "reads" the running memory state  $S_t$  to extract relevant historical information.

**Routing to the FFN:** Just like the SSM, the token-level outputs  $o_t$  are evaluated for all  $c$  steps and concatenated sequentially to form a full context matrix  $O \in \mathbb{R}^{c \times d_v}$ . Because the attention heads might have changed the vector width to  $d_v$ , this matrix is multiplied by a final output projection matrix  $W^O \in \mathbb{R}^{d_v \times d}$  to project it back to the expected hidden dimension  $d$ . This final  $c \times d$  matrix is what stands in place of  $\text{MHA}(Z)$ , continuing forward into Equation 14 to compute the intermediate state  $Z'$  for the Feed-Forward Network.

## 5 Network Stability Operations

### 5.1 Residual Connections

**Concept:** Adds the input of a layer directly to its output to prevent gradients from vanishing in deep networks[cite: 44]. Think of it as a highway bypass. If a complex sublayer transformation is failing or unnecessary early in training, the original signal can simply bypass it uncorrupted.

$$\text{Output} = x + \text{Sublayer}(x) \quad (13)$$

**Definitions:**

- $\text{Sublayer}(x)$ : A generic placeholder representing whichever transformation is currently being applied to  $x$  (e.g., MHA, SSM, or FFN)[cite: 45].

### 5.2 Dropout Regularization

$$\text{Dropout}(x) = x \odot \mathbf{m} \quad (14)$$

**Definitions and Mechanics:**  $\mathbf{m} \sim \text{Bernoulli}(1 - p)$  is a random mask vector containing 1s and 0s[cite: 46].  $p$  is the probability of a neuron being set to 0 (dropped)[cite: 47]. By randomly deactivating features during training, the network is forced to learn redundant, robust representations rather than relying too heavily on any single neuron (preventing "co-adaptation" or overfitting).

### 5.3 Layer Normalization (LayerNorm / RMSNorm)

Forces activations to have stable variance so that numbers do not grow infinitely large across many layers[cite: 48]. By keeping numerical ranges predictable, the loss landscape remains smooth and gradients do not explode.

$$\text{LayerNorm}(x) = \gamma \left( \frac{x - \mu}{\sqrt{\sigma_{var}^2 + \epsilon}} \right) + \beta, \quad \text{RMSNorm}(x) = \gamma \frac{x}{\sqrt{\frac{1}{d} \sum x_i^2 + \epsilon}} \quad (15)$$

**Definitions:**

- $\mu$  is the arithmetic mean (shifts the data to center around zero)[cite: 49].
- $\sigma_{var}^2$  is the variance (scales the data to have a spread of 1)[cite: 50].
- $\gamma, \beta \in \mathbb{R}^d$  are trainable scale and shift parameters. Once normalized, the network might actually *want* the data to be shifted or scaled for optimal performance;  $\gamma$  and  $\beta$  allow the network to explicitly relearn a safe shift and scale.
- $\epsilon$  is a tiny constant ( $10^{-5}$ ) to prevent division by zero[cite: 51].

### 5.4 The Intermediate State ( $Z'$ )

Applying these stability functions to the output of our sequence routing block (whether MHA,  $Y$  from Mamba, or  $O \cdot W^O$  from Gated Linear Attention) gives us the intermediate sequence matrix  $Z'$ :

$$Z' = \text{LayerNorm}(Z + \text{Dropout}(\text{RoutingModule}(Z))) \in \mathbb{R}^{c \times d} \quad (16)$$

—

## 6 Feed-Forward Network (FFN)

**Concept:** While Attention and SSMs route information *between* different tokens in the sequence, the Feed-Forward Network processes the information *within* each individual token[cite: 52]. It operates on each row of  $Z'$  independently and identically[cite: 53]. Its purpose is to project the token's representation into a higher-dimensional space where complex, non-linear features can be extracted, and then project it back down[cite: 54].

$$\text{FFN}(Z') = \text{ReLU}(Z'W_1 + b_1)W_2 + b_2 \in \mathbb{R}^{c \times d} \quad (17)$$

**Definitions:**

- $W_1 \in \mathbb{R}^{d \times d_{ff}}$ : The expansion weight matrix[cite: 55]. It projects the vector from dimension  $d$  up to  $d_{ff}$ .
- $W_2 \in \mathbb{R}^{d_{ff} \times d}$ : The contraction weight matrix[cite: 56]. It projects the vector back down to the standard embedding dimension  $d$ [cite: 57].
- $b_1 \in \mathbb{R}^{d_{ff}}, b_2 \in \mathbb{R}^d$ : Trainable biases.
- $\text{ReLU}(x)$ : Rectified Linear Unit, an activation function defined as  $\max(0, x)$ [cite: 58]. It converts all negative numbers to zero, injecting the necessary non-linearity into the network[cite: 59]. Without non-linearities, stacking multiple linear layers would collapse mathematically into just a single linear layer.

Applying the stability functions to the FFN completes one full architectural block, yielding  $Z_{out}$ :

$$Z_{out} = \text{LayerNorm}(Z' + \text{Dropout}(\text{FFN}(Z'))) \in \mathbb{R}^{c \times d} \quad (18)$$

This  $Z_{out}$  then becomes the input  $Z$  for the next layer[cite: 60]. This repeats  $k$  times.

## 7 Final Output and Loss Function

Once the sequence matrix has passed through all  $k$  network blocks, it must be mapped back into a language probability format so the model can make its prediction.

### 7.1 Logits and Probabilities

$$L = Z_k W_u \in \mathbb{R}^{c \times N}, \quad P = \text{Softmax}(L) \in \mathbb{R}^{c \times N} \quad (19)$$

**Definitions and Mechanics:**

- $Z_k \in \mathbb{R}^{c \times d}$ : The output of the final  $k$ -th block. This matrix holds the highly contextualized vector representations of all  $c$  tokens in the sequence.
- $W_u \in \mathbb{R}^{d \times N}$ : The un-embedding matrix. Also known as the language modeling head, it projects the  $d$ -dimensional hidden states back into the  $N$ -dimensional vocabulary space.
- $L \in \mathbb{R}^{c \times N}$ : The Logits matrix. These are the raw, unnormalized mathematical scores representing how strongly the model believes each word in the vocabulary is the correct next token.
- $P \in \mathbb{R}^{c \times N}$ : The predicted probability matrix. The Softmax function exponentiates the logits and normalizes them, ensuring that the predicted scores for all  $N$  words at any given position  $c$  sum to exactly 1.0.

## 7.2 Cross-Entropy Loss ( $\mathcal{L}$ )

To train the model, we must measure how far off its predicted probabilities  $P$  are from the actual correct words.

$$\mathcal{L} = - \sum_{j=1}^N Y_j \log(P_j) \quad (20)$$

### Definitions and Mechanics:

- $Y \in \mathbb{R}^N$ : The true target token (one-hot). Because the ground truth is one-hot encoded,  $Y_j = 1$  for the correct word's index, and  $Y_j = 0$  for all other indices in the vocabulary.
- $\log(P_j)$ : The natural logarithm of the predicted probability for the token at index  $j$ .
- **Mathematical Collapse:** Because  $Y_j$  is zero everywhere except at the correct token's index, the summation mathematically collapses to just a single term:  $\mathcal{L} = -\log(P_{\text{correct}})$ .
- **Objective:** By minimizing this negative log-likelihood loss, the optimizer is forced to push the predicted probability of the correct token ( $P_{\text{correct}}$ ) as close to 1.0 as possible.

—

## 8 Optimization (Updating the Weights)

### 8.1 The Trainable Parameters ( $\theta$ )

Let  $\theta$  represent the entire set of trainable parameters in the model[cite: 63]. This includes:

- $W_e, W_u$ : Embeddings.
- $W_i^Q, W_i^K, W_i^V, W^O$ : Attention projections (or their SSM equivalents)[cite: 64].
- $W_1, W_2, b_1, b_2$ : FFN weights and biases.
- $\gamma, \beta$ : Normalization scale and shift parameters[cite: 65].

### 8.2 Backpropagation (The Chain Rule)

To update any weight matrix  $W \in \theta$ , we calculate its gradient[cite: 66]. For a standard linear layer where  $Y = X \cdot W$ :

$$\nabla_W \mathcal{L} = X^T \cdot \frac{\partial \mathcal{L}}{\partial Y} \quad (21)$$

### Definitions:

- $\nabla_W \mathcal{L}$  is the gradient of the loss with respect to  $W$ [cite: 67]. It points in the direction of steepest ascent (where the error increases the most).
- $\frac{\partial \mathcal{L}}{\partial Y}$  is the error signal passed backward from the layers ahead[cite: 68]. Using the chain rule of calculus, the network works backward from the final loss, figuring out exactly how much each parameter contributed to the mistake.

### 8.3 Adam Optimizer

Adam adjusts the parameters  $\theta$  by tracking moving averages of the gradients[cite: 69].

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) \nabla_{\theta} \mathcal{L}, \quad v_t = \beta_2 v_{t-1} + (1 - \beta_2) (\nabla_{\theta} \mathcal{L})^2 \quad (22)$$

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t} \quad (23)$$

$$\theta_{t+1} = \theta_t - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon} \quad (24)$$

#### Definitions and Physical Intuition:

- $\eta$ : The Learning Rate (the step size taken in the direction of the gradient)[cite: 70].
- $m_t$ : First moment (momentum). Tracks the general direction the gradient is pointing. Like a ball rolling down a hill, it builds up speed if it keeps moving in the same direction, helping it power through small bumps (local minima).
- $v_t$ : Second moment (velocity/variance)[cite: 71]. Tracks how violently the gradient is oscillating[cite: 72]. If the gradient jumps wildly back and forth across a ravine,  $v_t$  gets large. Dividing by  $\sqrt{v_t}$  artificially applies "friction," slowing the optimizer down to prevent it from ping-ponging out of control.
- $\hat{m}_t, \hat{v}_t$ : Bias-corrected versions of  $m_t$  and  $v_t$  to prevent initial updates from skewing toward zero[cite: 73].
- $\beta_1, \beta_2$ : Exponential decay rates for the moment estimates[cite: 74]. Typically 0.9 and 0.999, controlling how quickly old gradients are forgotten.
- $\epsilon$ : A tiny constant to prevent division by zero in the denominator.

### 8.4 Muon Optimizer & Newton-Schulz Iteration

An alternative optimizer specifically designed for internal weight matrices ( $W$ )[cite: 75].

$$W_{t+1} = W_t - \eta \cdot \text{NewtonSchulz}(\nabla_W \mathcal{L}) \quad (25)$$

**Definitions and Mechanics:**  $\text{NewtonSchulz}(M)$  mathematically transforms an input matrix  $M$  into an orthogonal matrix  $Q$  (where  $Q^T Q = I$ , the identity matrix)[cite: 76]. It is approximated iteratively via:

$$M_{k+1} = \frac{1}{2} M_k (3I - M_k^T M_k) \quad (26)$$

This orthogonalization forces the gradient updates to be mathematically independent across rows and columns. In practice, this prevents the network from learning redundant, overlapping features across different attention heads, making training highly efficient[cite: 77].

## 9 Typical Hyper-parameter Values

These are standard configurations balancing memory capacity and computational cost.

| Hyper-parameter         | Symbol   | Typical Value(s)                                         |
|-------------------------|----------|----------------------------------------------------------|
| Embedding Dimension     | $d$      | 512, 1024, 4096 (Larger = more expressive power)         |
| Number of Heads         | $h$      | 8, 32, 128 (More heads = finer granular routing)         |
| Head Dimension          | $d_k$    | 64 or 128 [cite: 78]                                     |
| Context Size            | $c$      | 4096, 128k, 1M+ (Standard MHA struggles past 8k context) |
| Number of Blocks        | $k$      | 12, 24, 96 [cite: 79]                                    |
| FFN Hidden Dimension    | $d_{ff}$ | $4 \times d$ (Standard expansion ratio)                  |
| SSM State Dimension     | $n$      | 16, 64, 128                                              |
| Dropout Rate            | $p$      | 0.0 to 0.1 (10%) [cite: 80]                              |
| Adam Learning Rate      | $\eta$   | $1 \times 10^{-4}$ to $6 \times 10^{-4}$                 |
| Adam $\beta_1, \beta_2$ | -        | 0.9, 0.95 or 0.999                                       |
| Vocabulary Size         | $N$      | 32,000 to 128,000 [cite: 81]                             |